SEND

An actor can only communicate with another actor by sending it messages. This takes *encapsulation* to the next level. With objects, you can specify which methods can be publicly called and which state is accessible from the outside. Actors don't allow access to internal state (for example, the list of messages in a conversation). Actors can't share mutable state; they can't, for instance, point to a shared list of conversation messages and change the conversation in parallel at any point in time.

The `Conversation` actor can't simply call a method on any other actor, since doing so could lead to sharing mutable state. It has to send the other actor a message. Sending messages is always asynchronous in what is called a *fire-and-forget* style. If it's important to know that another actor received the message, the receiving actor should send back an acknowledgment message.

The `Conversation` actor doesn't have to wait and see what happens with a message to the `Mentions` actor; it can send a message and continue its work. Asynchronous messaging helps the chat application decouple the components; this is one reason you used a message queue for the `Mentions` object, which is now unnecessary.

The messages need to be immutable, meaning they can't be changed once they're created. This makes it impossible for two actors to change the same message by mistake or for a single actor to change a message twice but in different ways, both of which could result in unexpected behavior.